

## Práctica Individual 1 – Ejercicios iterativos, recursivos y notación funcional. Análisis empírico de la complejidad.

### EJEMPLOS:

1. Un punto es un tipo con las siguientes propiedades:

- X, Double, básica, individual
- Y, Double, básica, individual
- Cuadrante, Cuadrante, derivada, individual. Enumerado {PRIMER\_CUADRANTE, SEGUNDO\_CUADRANTE, TERCER\_CUADRANTE, CUARTO\_CUADRANTE}.

```
public static Map<Cuadrante, Double> solucionFuncional (List<Punto2D> ls) {
    return ls.stream()
        .collect(Collectors.groupingBy(Punto2D::cuadrante,
            Collectors.reducing(0., p -> p.x(), (a, b) -> a + b)));
}
```

Analice el código que se muestra y proporcione una solución iterativa y otra recursiva final equivalentes.

2. Dada la siguiente definición recursiva de la función  $f$  (que toma como entrada 2 números enteros positivos y devuelve una cadena):

$$f(a, b) = \begin{cases} "(" + \text{toString}(a * b) + ")", & a < 5 \vee b < 5 \\ \text{toString}(a + b) + f(a/2, b - 2), & \text{en otro caso} \end{cases}$$

siendo  $+$  un operador que representa la concatenación de cadenas, y  $\text{toString}(i)$  un método que devuelve una cadena a partir de un entero. Proporcione una solución iterativa usando `while`, una recursiva no final, una recursiva final, y una en notación funcional.

3. Dada la siguiente definición recursiva de la función  $g$  (que toma como entrada 2 números enteros positivos y devuelve un entero):

$$g(a, b) = \begin{cases} a^2 + b, & a < 2 \vee b < 2 \\ g\left(\frac{a}{2}, b - 1\right) + g\left(\frac{a}{3}, b - 2\right), & \text{en otro caso} \end{cases}$$

Proporcione una solución recursiva sin memoria, otra recursiva con memoria, y otra iterativa.

4. Implementar de forma recursiva e iterativa un algoritmo para el cálculo de la potencia  $a^n$ , siendo  $a$  de tipo `Double` y  $n$  de tipo `Integer`. Analizar sus tiempos de ejecución.

5. Analizar los tiempos de ejecución de las versiones recursivas sin y con memoria, iterativa y haciendo uso de la potencia de matrices para el cálculo de los números de Fibonacci. Comparar según los resultados sean de tipo `Double` o `BigInteger`.

**EJERCICIOS:**

1. Analice el código que se muestra a continuación, en el que EnteroCadena es una clase con una propiedad entera `a` y otra de tipo cadena `s` (la cual debe implementar como un record):

```
public static String ejercicio1 (Integer varA, Integer varB) {
    UnaryOperator<EnteroCadena> nx = elem ->
    {
        return EnteroCadena.of(elem.a()+3,
                               elem.a()%2==0?
                               elem.a()+"*":
                               elem.a()+"!");
    };

    return Stream
        .iterate(EnteroCadena.of(varA,"A"), elem -> elem.a() < varB, nx)
        .filter(elem -> elem.a()%10 != 0)
        .map(elem -> elem.s())
        .collect(Collectors.joining("-"));
}
```

**SE PIDE:** Proporcione una solución iterativa y otra recursiva final equivalentes.

2. Dada la siguiente definición recursiva de la función  $f$  (que toma como entrada un número entero positivo y una cadena, y devuelve una lista de cadenas):

$$f(a,s) = \begin{cases} [toString(a)+s], & a \leq 2 \vee s.length \leq 2 \\ f\left(\frac{a}{2}, s.substring(0, s.length - 2)\right) + toString(a), & a \text{ es par} \\ f\left(\frac{a}{3}, s.substring(0, s.length - 1)\right) + (toString(a)+s.substring(0, a\%s.length)), & eoc \end{cases}$$

donde:

- `[elem]`: lista de un único elemento `elem`
- `s1+s2`: concatenación de las cadenas `s1` y `s2`
- `list + elem`: inserción del elemento `elem` al final de la lista `list`
- `toString(a)`: cadena asociada al entero `a`.
- `s.substring(inicio,fin)`: subcadena de la cadena `s`, incluyendo los caracteres de `s` en el rango `[inicio, fin)`.

**SE PIDE:** Proporcione una solución recursiva no final, una recursiva final, una iterativa usando `while`, y una en notación funcional.

3. Dada la siguiente definición recursiva de la función  $g$  (que toma como entrada 3 números enteros positivos y devuelve un **conjunto** de enteros):

$$g(a,b,c) = \begin{cases} \{a+2, b*3, c\}, & a \leq 5 \vee b \leq 3 \vee c \leq 2 \\ \{a, b+2, c*3\}, & a \leq 8 \vee b \leq 6 \vee c \leq 4 \\ g\left(\frac{a}{2}, b-3, c/4\right) + g\left(a-3, \frac{b}{3}, c-2\right), & eoc \end{cases}$$

siendo  $+$  un operador que representa la unión de conjuntos.

**SE PIDE:** Proporcione una solución recursiva sin memoria, otra recursiva con memoria, y otra iterativa.

4. Dada la siguiente definición recursiva de la función  $f$  (que toma como entrada un número entero positivo):

$$h(a) = \begin{cases} 5, & a < 10 \\ \text{Math.sqrt}(3 * a) * h(a - 2), & \text{en otro caso} \end{cases}$$

**SE PIDE:** Analizar los tiempos de ejecución de la versiones recursiva e iterativa de dicha función, comparando según los resultados sean de tipo Double o BigInteger. Para ello, debe obtener 5 gráficas:

- La que se obtiene cuando los resultados son de tipo Double usando un esquema recursivo.
- La que se obtiene cuando los resultados son de tipo BigInteger usando un esquema recursivo.
- La que se obtiene cuando los resultados son de tipo Double usando un esquema iterativo.
- La que se obtiene cuando los resultados son de tipo BigInteger usando un esquema iterativo.
- La que se obtiene combinando las 4 anteriores.

5. Dada la siguiente definición recursiva de la función  $f$  (que toma como entrada un número entero positivo):

$$k(n) = \begin{cases} 1, & n \leq 6 \\ 1 + k(n - 1) * \log_2(n - 1), & n > 6 \end{cases}$$

```
public static int log2(int n){  
    if(n <= 0) throw new IllegalArgumentException();  
    return 31 - Integer.numberOfLeadingZeros(n);  
}
```

**SE PIDE:** Analizar los tiempos de ejecución de las versiones recursiva e iterativa de dicha función, comparando según los resultados sean de tipo Double o BigInteger. Para ello, debe obtener 5 gráficas:

- La que se obtiene cuando los resultados son de tipo Double usando un esquema recursivo.
- La que se obtiene cuando los resultados son de tipo BigInteger usando un esquema recursivo.
- La que se obtiene cuando los resultados son de tipo Double usando un esquema iterativo.
- La que se obtiene cuando los resultados son de tipo BigInteger usando un esquema iterativo.
- La que se obtiene combinando las 4 anteriores.

**Debe resolver todos los ejercicios de forma eficiente. Tenga en cuenta que:**

- Para cada ejercicio debe leer los datos de entrada de un fichero, y mostrar la salida por pantalla. Dicha lectura debe ser independiente del algoritmo concreto que resuelva el ejercicio.
- La solución tiene que ser acorde al material de la asignatura proporcionado.